

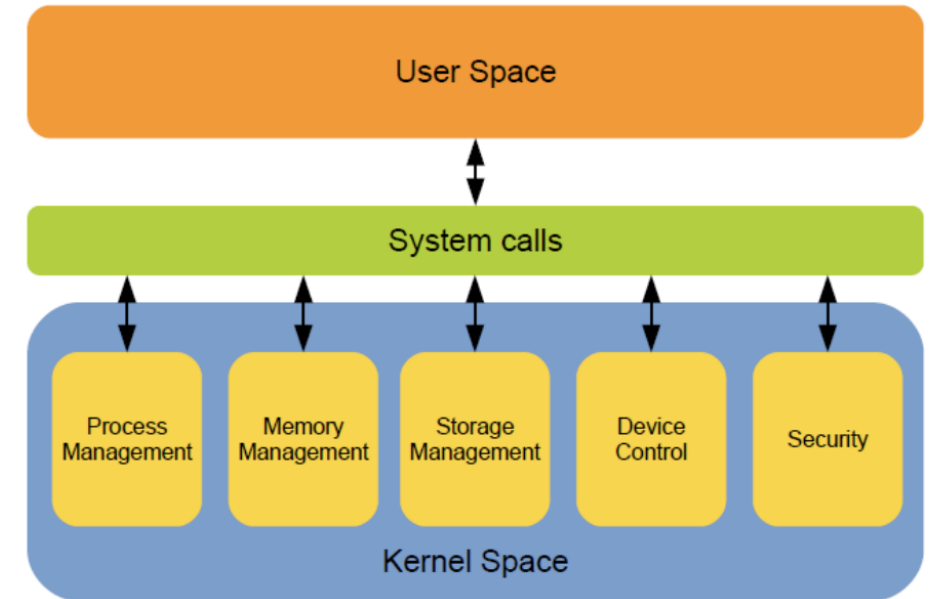


Mass-Storage Structure

Ivy Peng
Associate Professor
EECS, KTH

OS Organization

- Basic organization of an Operating System
 - Process management
 - Memory management
 - **Storage management and File system**
 - Device control and I/O subsystem
 - Protection and security

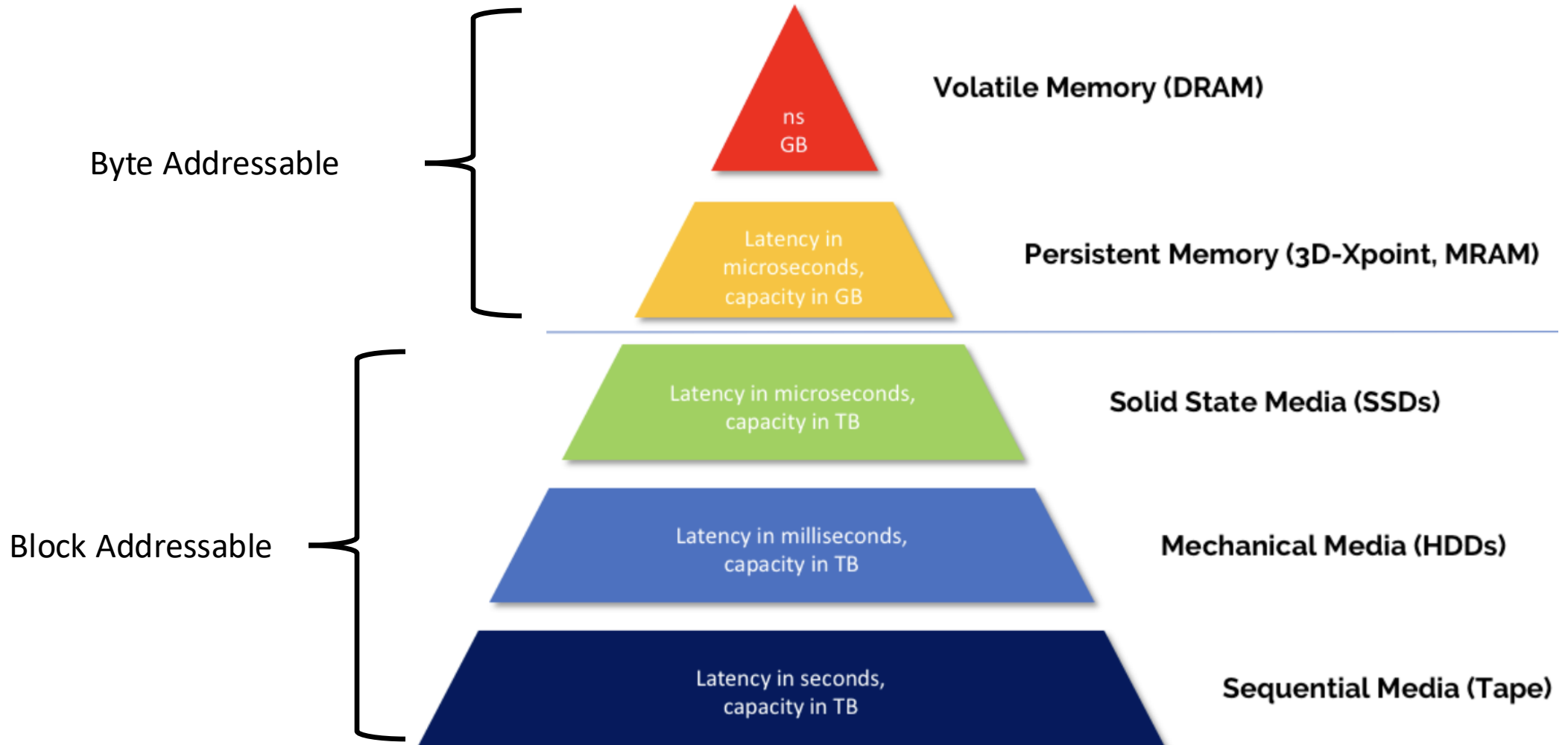




Outline

- **Storage Devices**
- Hard disk drives (HDD) Scheduling
- Non-volatile memory (NVM) Scheduling
- Disk Attachment
- RAID Structure

The Memory-Storage Hierarchy



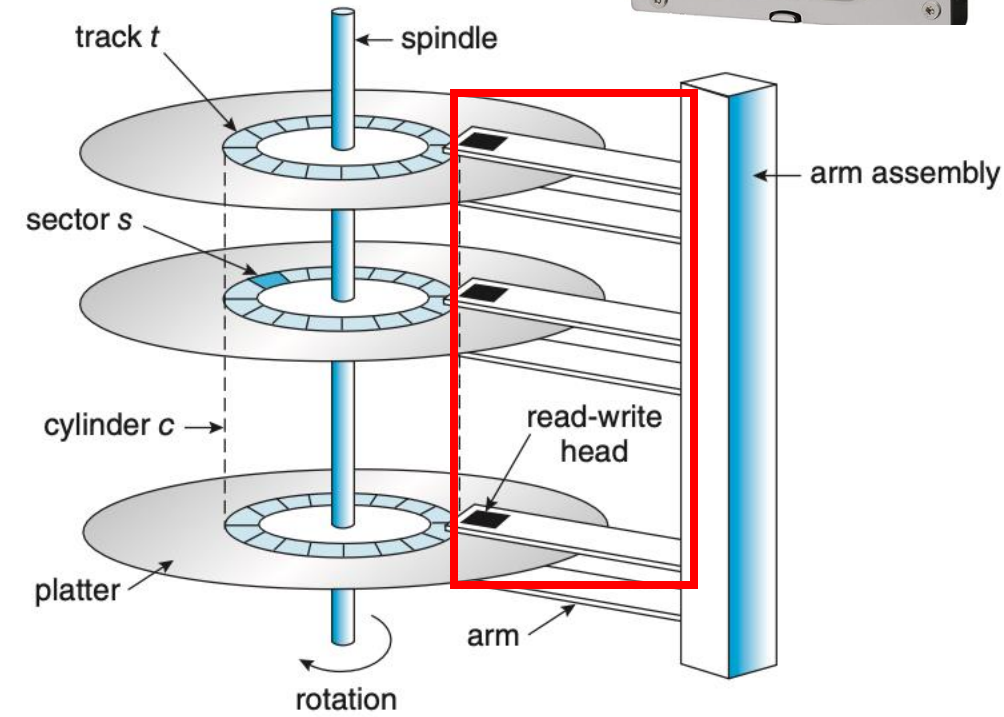
Secondary Storage

- Main memory is usually too small
- Computer systems must provide secondary storage to back up main memory.
 - Hard Disk Drives (HDD)
 - Solid-State Disks (SSD)
- Other slower and larger storage includes magnetic tape, optical disks, etc
- One useful metric: byte/\$



Hard Disk Drives (HDD)

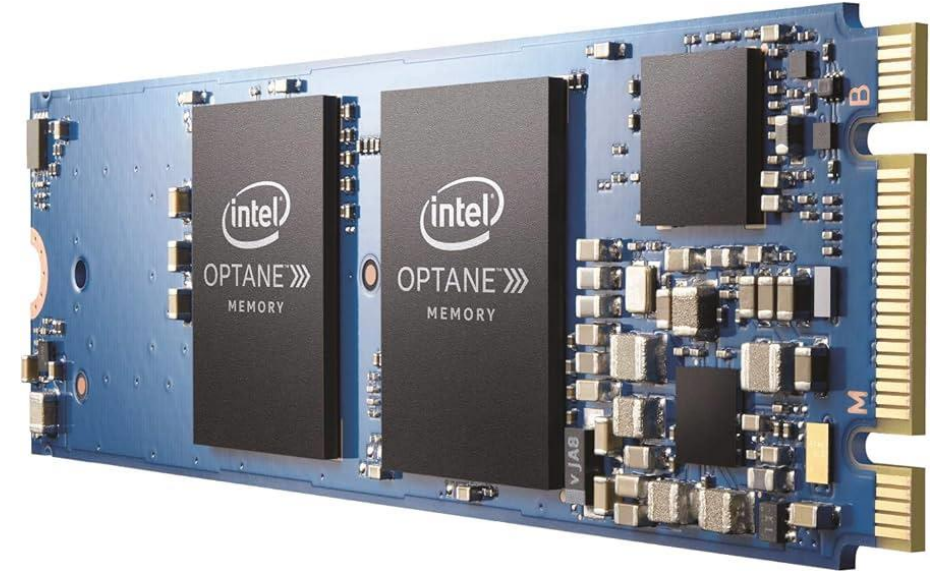
- Typical capacity in terabytes (TB)
- Organized into:
 - **Platter**: two surfaces with magnetic materials to store information
 - Track
 - Sector: the smallest unit of data transfer, e.g., 512 bytes, 4KB
- Performance is measured in **data transfer rate**, typically in hundreds of MB/s
- Data Transfer Rate depends on:
 - How fast the spindle rotates, i.e., rotation per minutes (RPM)
 - Seek time
 - Rotational latency



HDD moving-head disk mechanism.

Non-volatile Memory (NVM) Devices

- NVM devices consist of:
 - a **controller** : schedule commands
 - Non-volatile semiconductor **chips** : where data is stored in 'semiconductor pages'
- A lot of different NVM technologies:
 - Flash Memory, such as NAND flash
 - 3D Xpoint: Intel Optane
 - Emerging byte-addressable technologies like ReRAM, PCM, etc.
- Flash memory is the most common:
 - SSD is a form of flash memory NVM in disk-drive container
 - USB drive is a form of flash memory with USB interface



Intel Optane Memory M10 Series (64GB, 3D XPoint) ...



Non-volatile Memory Devices

- Faster than HDD
- Less capacity than HDD
- More reliable than HDD as no parts are moving or rotating
- More expensive (byte/\$)
- Shorter life span than HDD



Connection Methods

- Storage devices, as a type of peripheral devices, is often connected to a computer by I/O bus:
 - Serial ATA (SATA) is mostly common for connecting hard drive
 - PCIe is used to connect NVMe SSD

Connecting a SATA HDD

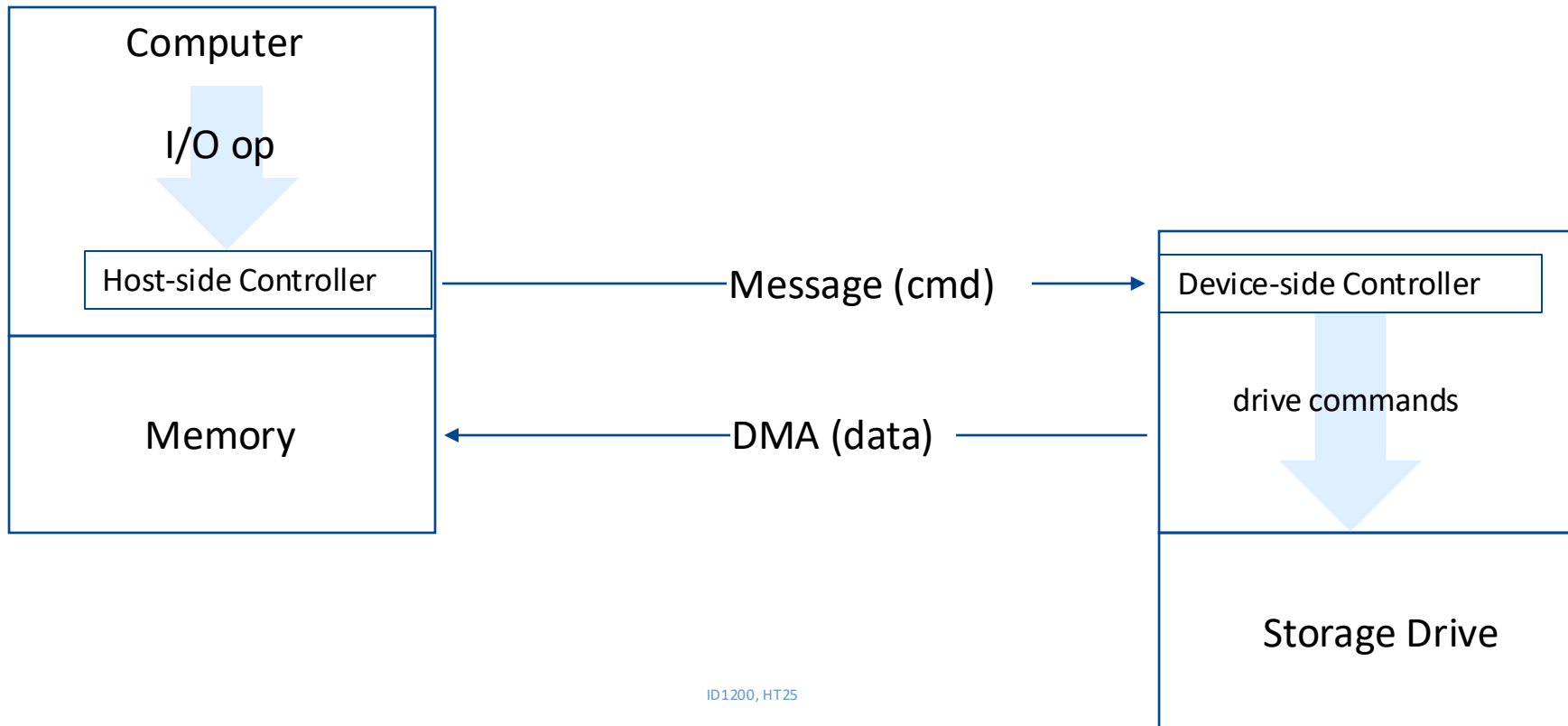


NVMe PCIe SSD card



Connection Methods

- A simplified workflow for issuing an I/O operation from Computer
- Host Controller sends command in message to Device Controller
- Device Controller schedules/issues drive commands to drive hardware, e.g., NVMe commands
- Data fetched from storage device is copied to memory via DMA



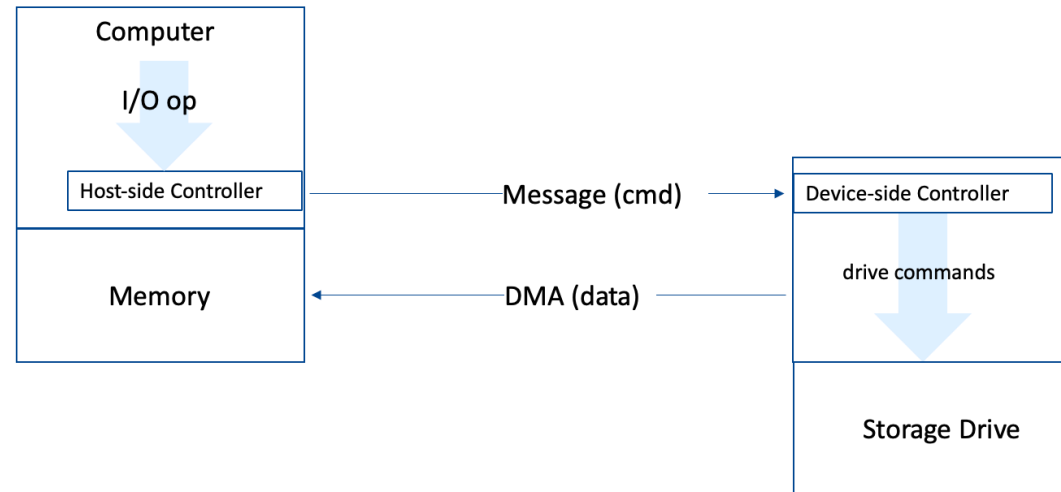


Outline

- Storage Devices
- **Hard disk drives (HDD) Scheduling**
- Non-volatile memory (NVM) Scheduling
- Disk Attachment
- RAID Structure

HDD Scheduling

- There are many sources of disk I/O requests, e.g., OS, system processes, user processes.
- OS maintains **queue of requests**, per disk or device.
- Idle disk can immediately work on I/O request
- Busy disk means that requests will be put in the queue.



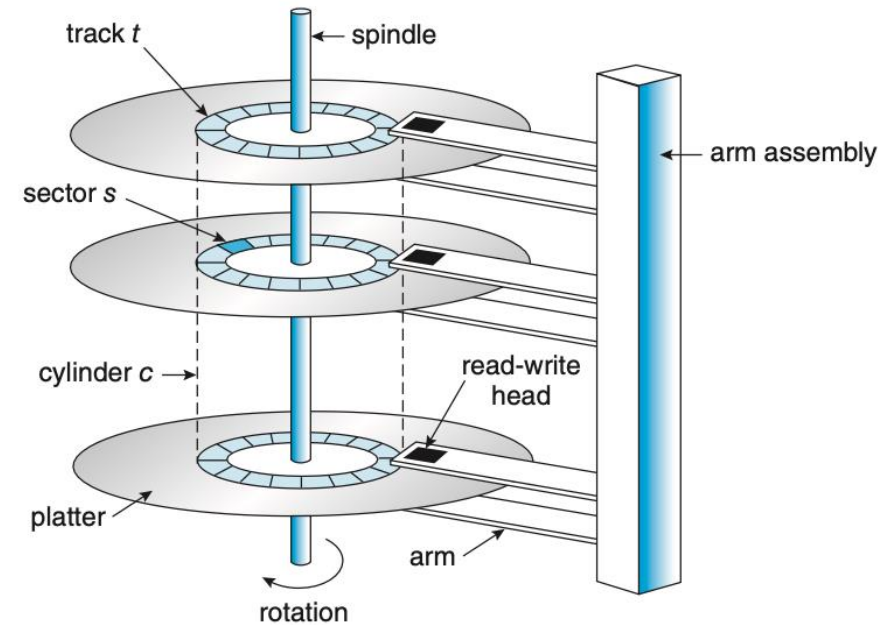
HDD Scheduling

- For multiprogramming, there are often pending requests in the device queue
- Performance optimization can be achieved via reordering requests
 - More sequential I/O
 - More timely response
 - Fairness

Queue Depth and commands		
Protocol	Queue Depth	Commands
SAS	1	254
SATA (AHCI)	1	32
NVMe	65,000	65,000

I/O Requests to HDD

- Access time is determined by **Seek** time and **Rotation** latency
 - Seek time is for moving the head to the cylinder containing the desired sector
 - Rotation latency is to rotating the desired sector to the head
- For multiple pending requests, exploit the locality among them may reduce the total access time

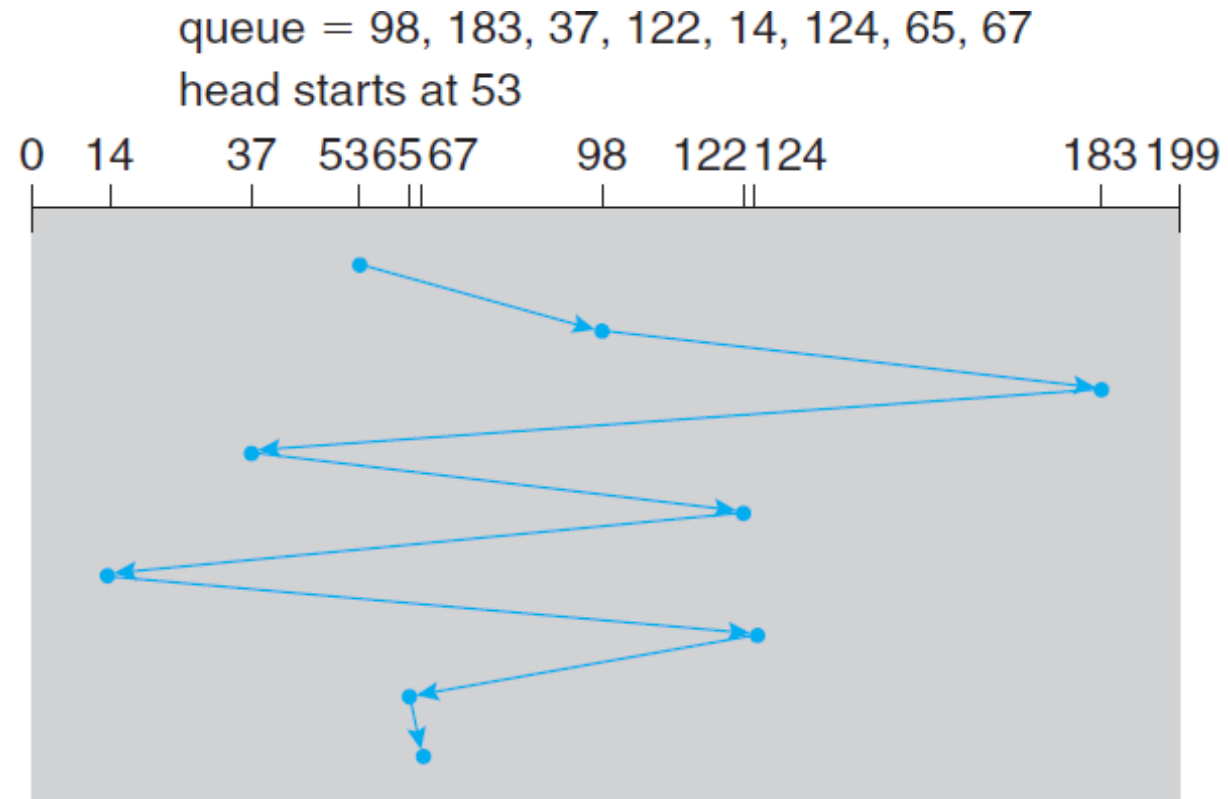




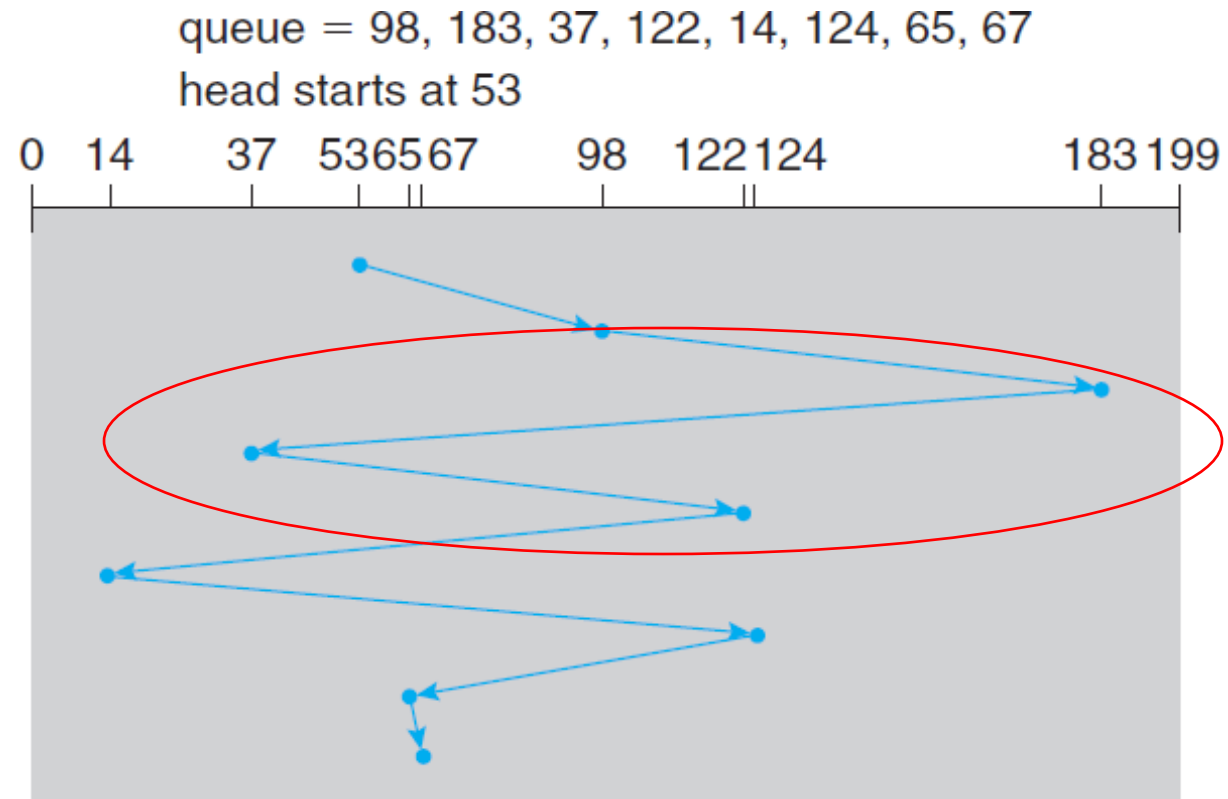
HDD Scheduling Algorithms

- First Come First Serve (FCFS)
- Shortest Seek Time First (SSTF)
- SCAN
- C-SCAN
- C-Look

- Request queue (0-199): 98, 183, 37, 122, 14, 124, 65, 67
- Head pointer 53
- **Total head movement:** 640 cylinders



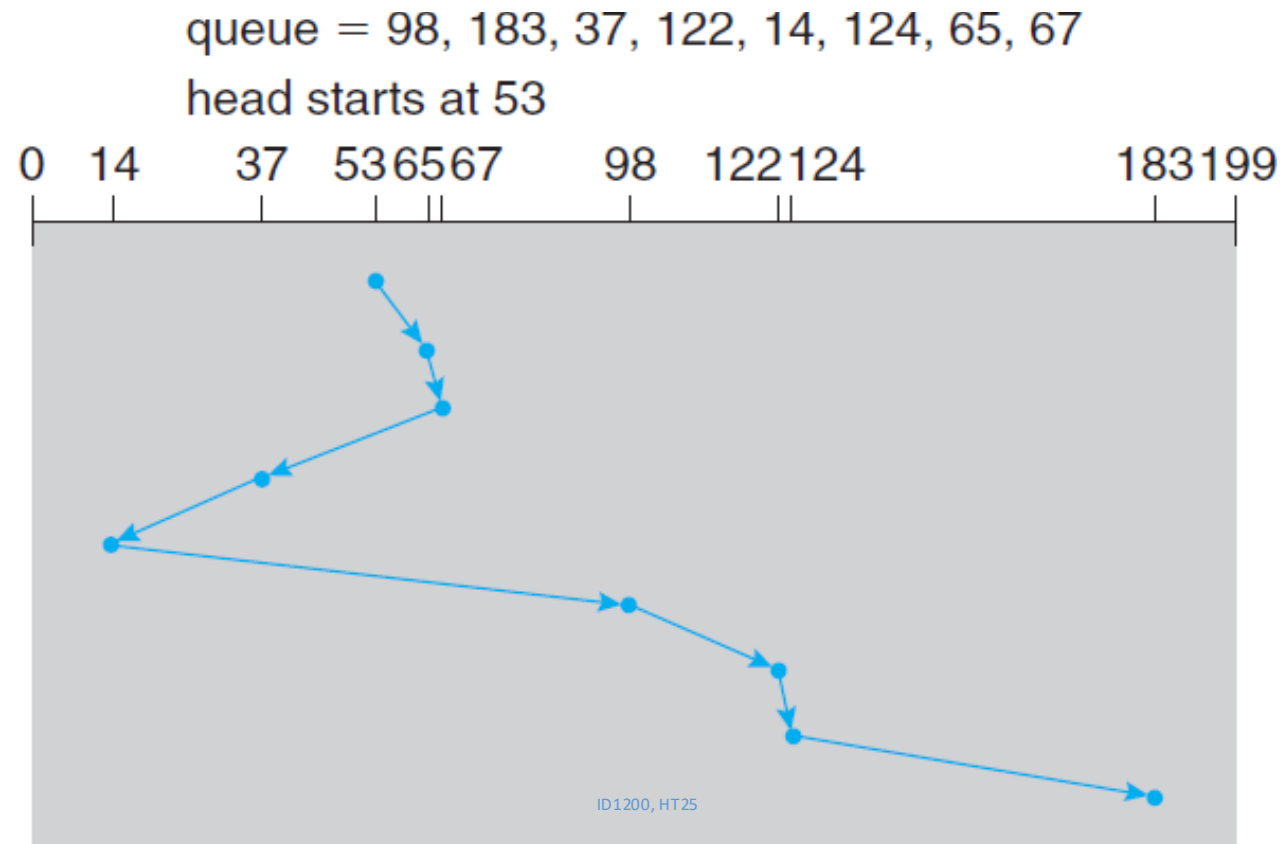
- Request queue (0-199): 98, 183, 37, 122, 14, 124, 65, 67
- Head pointer 53
- **Total head movement:** 640 cylinders



$(98-53)+$
 $(183-98)+$
 $(183-37)+$
 $(122-37)+$
 $(122-14)+$
 $(124-14)+$
 $(124-65)+$
 $(67-65)$

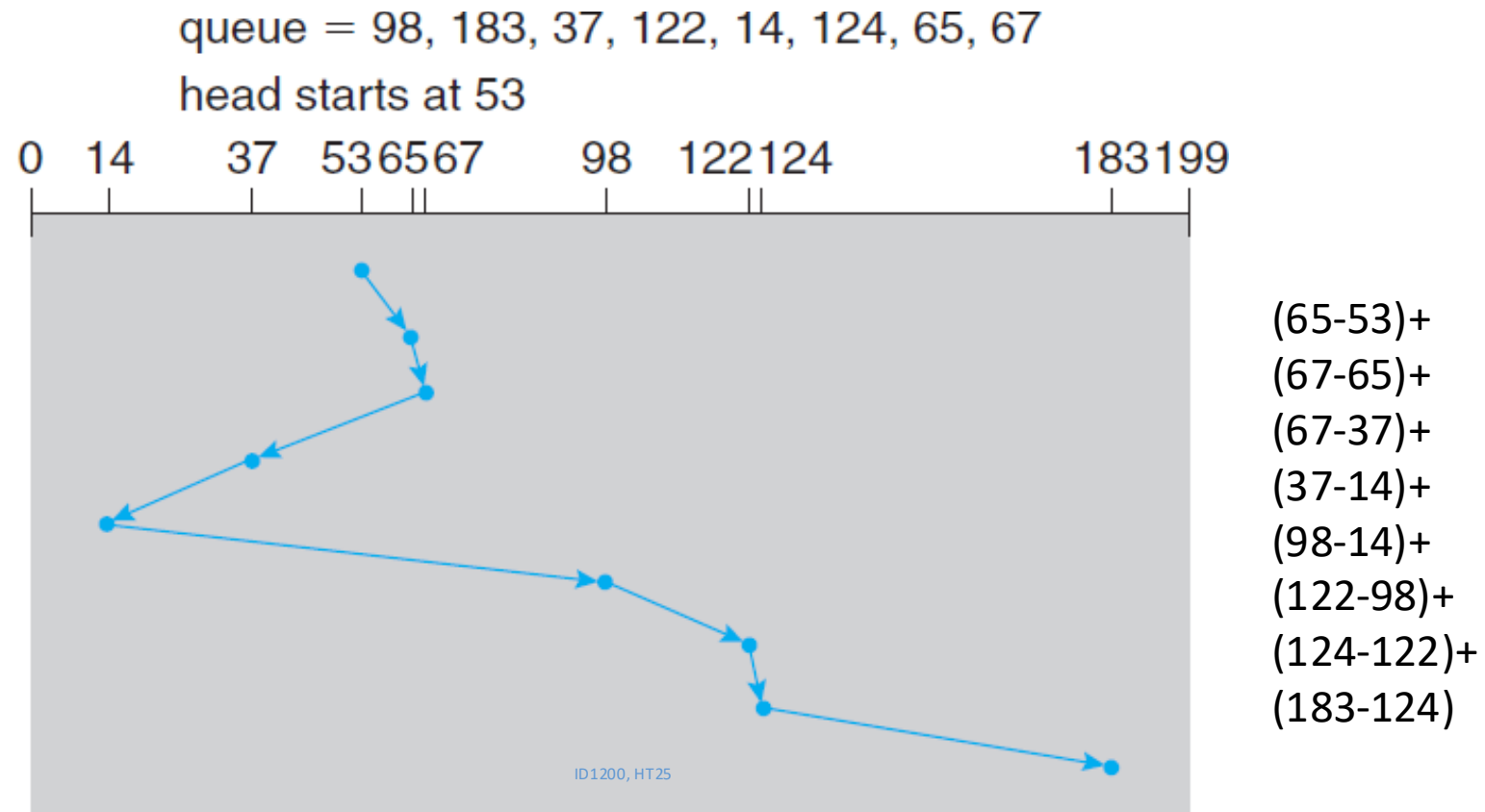
Shortest Seek Time First (SSTF)

- Selects the request with the minimum seek time from the current head position.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.
- **Total head movement:** 236 cylinders.

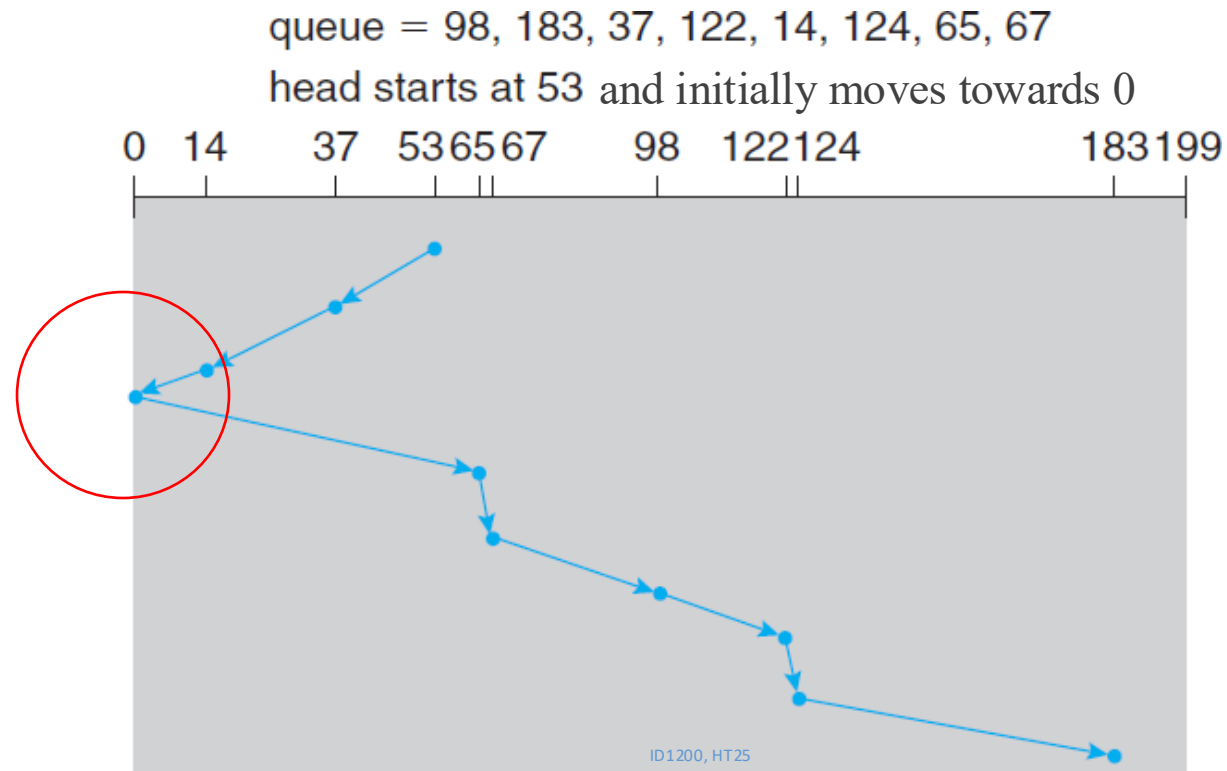


Shortest Seek Time First (SSTF)

- Selects the request with the minimum seek time from **the current head** position.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.
- **Total head movement:** 236 cylinders.



- Starts from **the current head** position and moves in **the direction of head movement**.
- Servicing requests on the direction of movement until it gets to the other end of the disk.
- At the end of the disk, the head movement is reversed.
- Total head movement: 236 cylinders



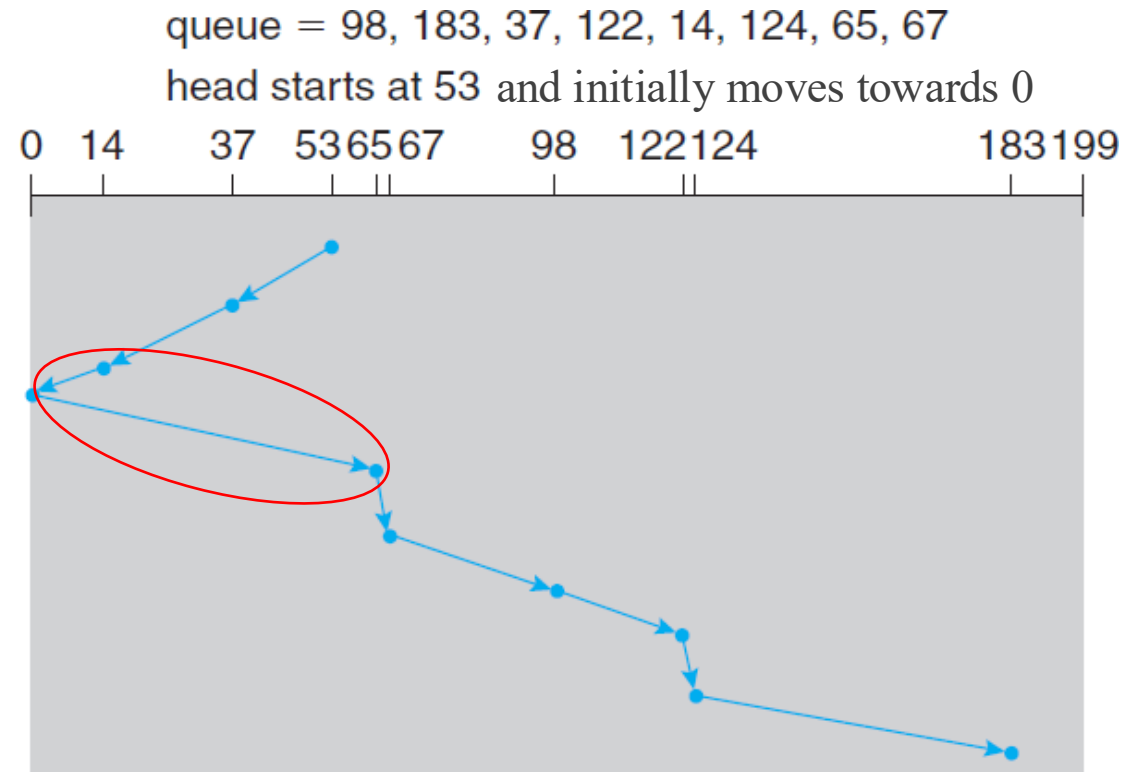
- Starts from **the current head** position and moves in **the direction of head movement**.
- Servicing requests on the direction of movement until it gets to the other end of the disk.
- At the end of the disk, the head movement is reversed.
- Total head movement: 236 cylinders

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53 and **initially moves towards 199 ?**

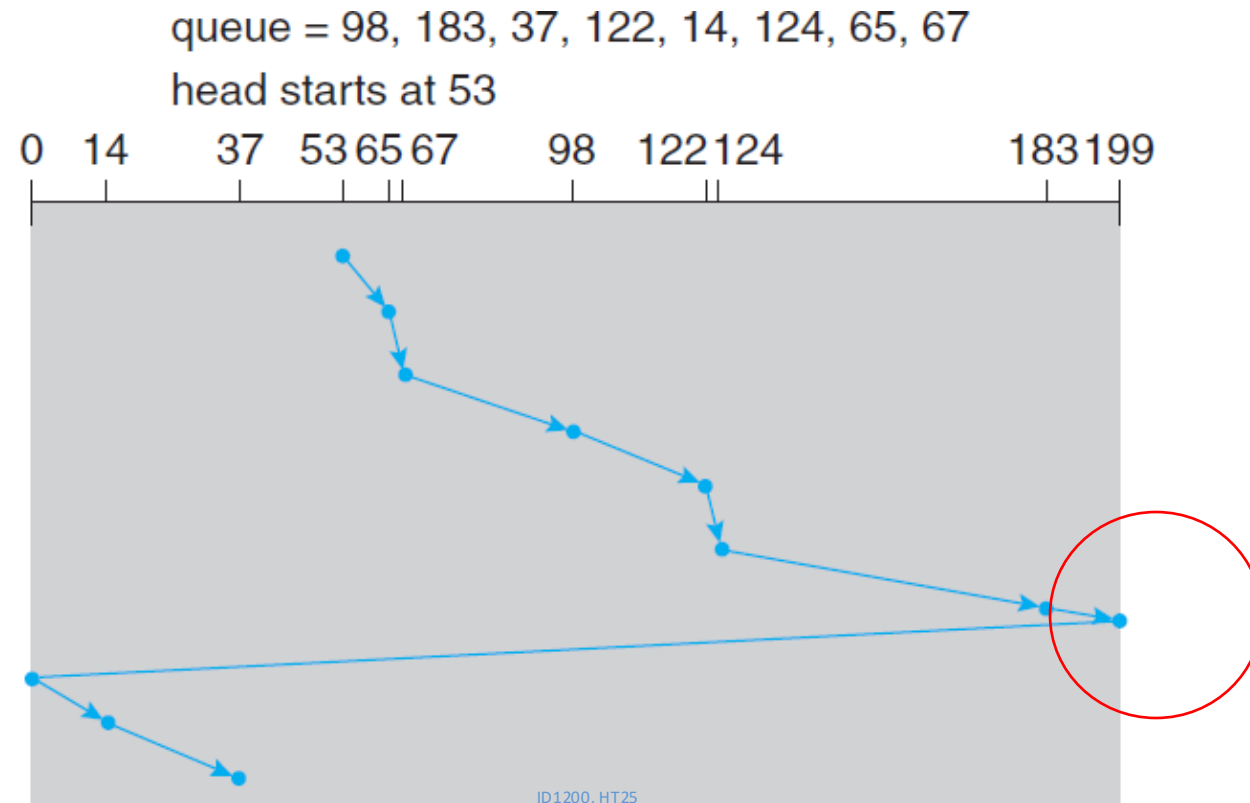
0 14 37 53 65 67 98 122 124 183 199

- Limitations: when reversed, it's less likely to have requests as they have been recently serviced



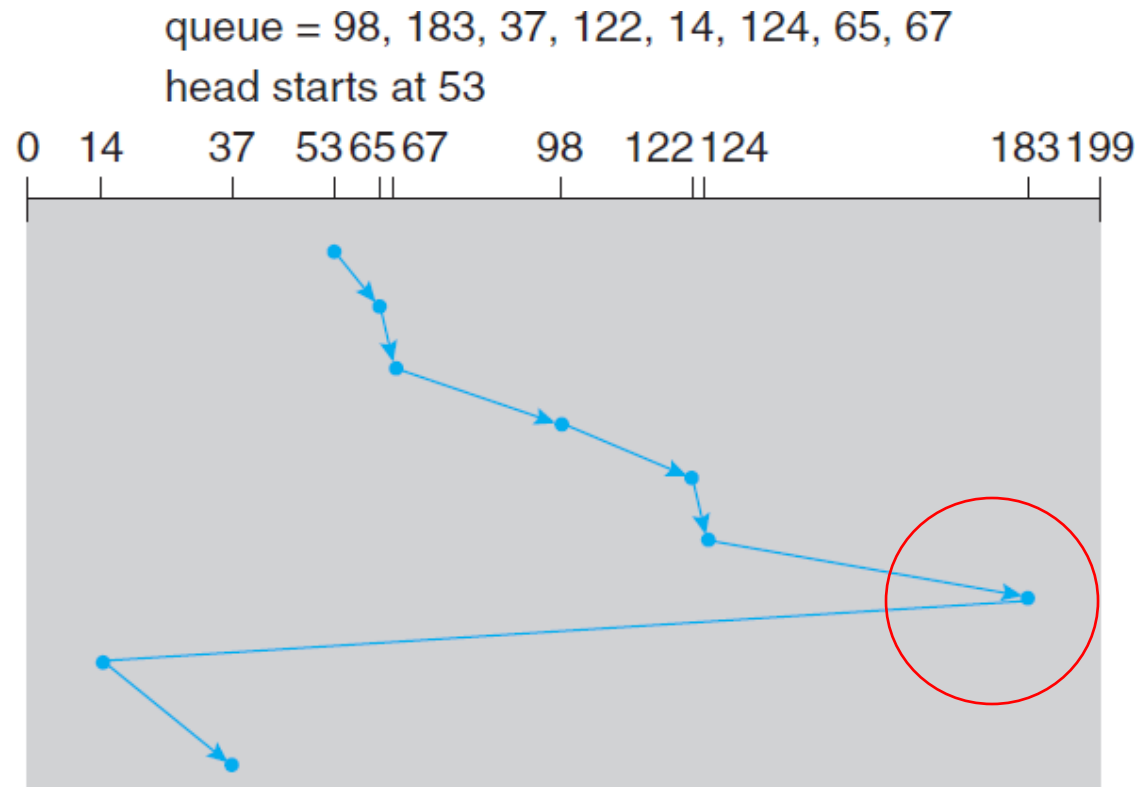
C-SCAN

- An improved version of SCAN
- Provides a **more uniform wait time** than SCAN.
- When it reaches the end, it immediately returns to the beginning of the disk without servicing any requests on the return trip.



LOOK and C-LOOK

- LOOK is a version of SCAN, C-LOOK is a version of C-SCAN.
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.



Select a Scheduling Algorithm (1/2)

- Metrics: Access Time and Disk Bandwidth
- Minimize the seek time : given a sequence of pending requests in the queue, which scheduling algorithms, FCFS, SSTF, SCAN, C-SCAN, C-Look, to use?
- Disk Bandwidth = $\frac{\text{Total Bytes Transferred}}{\text{Total Time}}$
 - Here the total time is the time interval between the first request and the completion of the last transfer.

Select a Scheduling Algorithm (2/2)

- Performance depends on the number and types of requests.
- SSTF is common and often yields good performance
 - It minimize seek time
- SCAN and C-SCAN perform better for systems that place a heavy load on the disk
 - less starvation, more fairness



Outline

- Storage Devices
- Hard disk drives (HDD) Scheduling
- **Non-volatile memory (NVM) Scheduling**
- Disk Attachment
- RAID Structure

- NVM is better for **random access I/O** than HDD
 - It does not have the mechanical moving parts
 - No need to minimize seek time as in HDD.
- Read Access time is mostly uniform. FCFS is often sufficient.
- Performance as in **random access I/O** is measured in input/output operations per second (**IOPS**)
 - HDDs in 100s IOPS
 - NVMs in 100,000s IOPS



- Write access time, is worse than Read, and is more dynamic.
 - Depend on how **full** and how **worn** is the drive
- Data cannot be overwritten, need to erase the page first and then write to it
 - Every erase wears out the drive
 - After a number of erase, drive cannot retain data anymore
- The life span of NVM depends how worn each block is, the more writes to a block, the shorter life span it has
- Most optimization on NVM focuses on **writes** to improve performance and lifespan



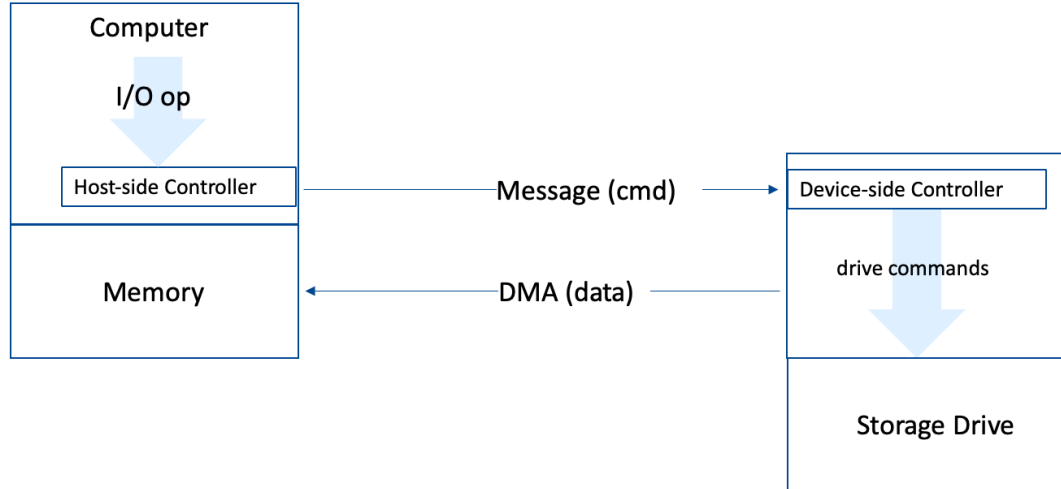
HDD v.s NVM (NAND Flash Memory)



Dong Ngo/CNET: <https://www.cnet.com/tech/computing/digital-storage-basics-part-4-ssd-explained/>

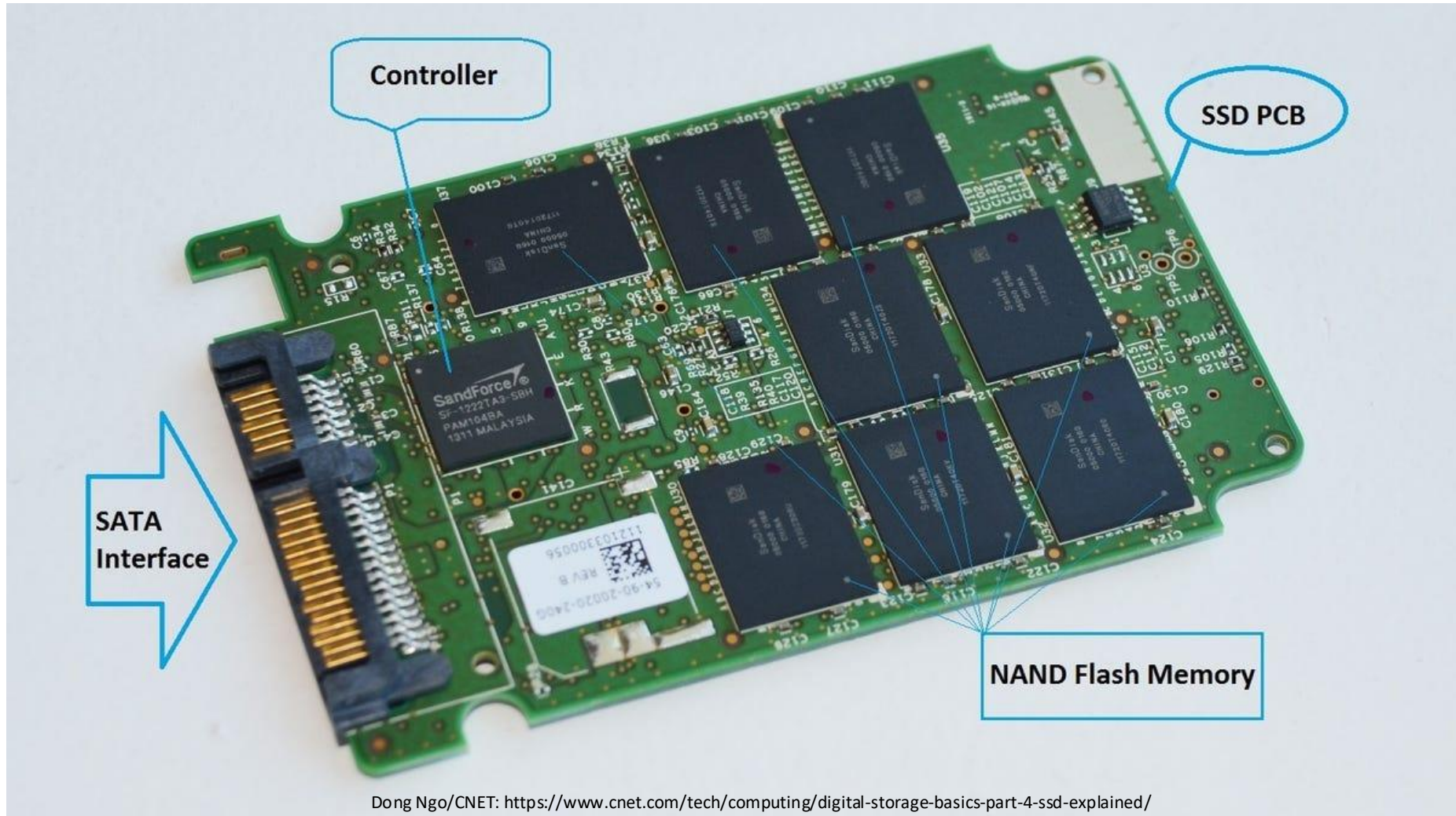
NVM Scheduling

- Similar to HDD, there are often pending requests in the device queue
- Recent NVMe SSD devices can hold very deep command queues
- Reordering requests can help improve performance (write) and lifespan (wear leveling)
- Most optimization is done within device controller



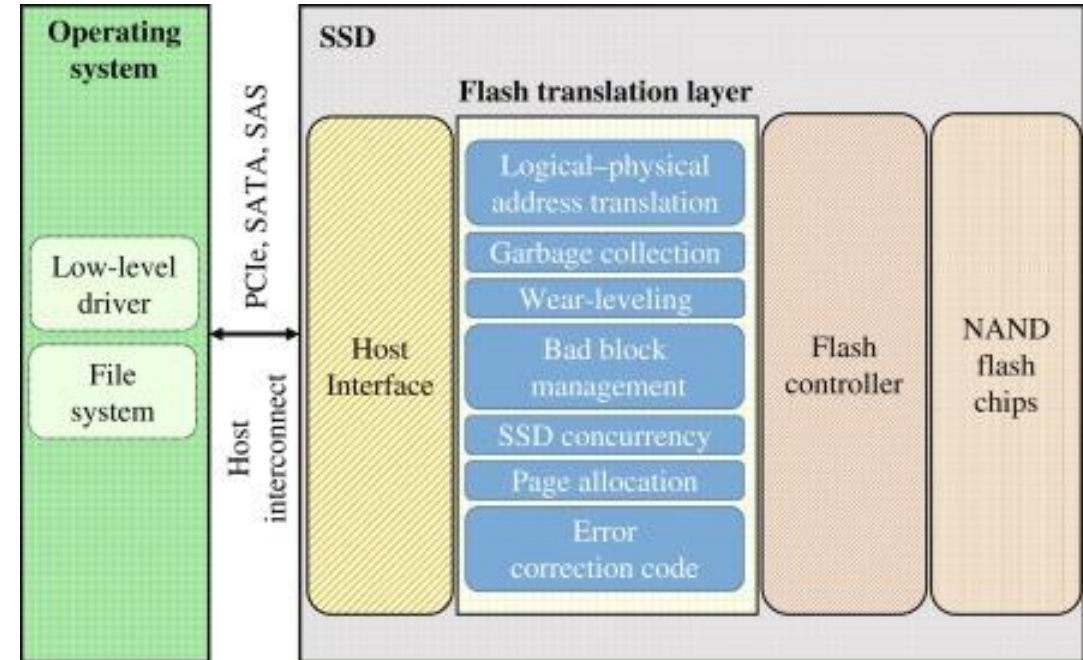
Queue Depth and commands		
Protocol	Queue Depth	Commands
SAS	1	254
SATA (AHCI)	1	32
NVMe	65,000	65,000

NVM (NAND Flash Memory)



Case study: NAND Flash Memory

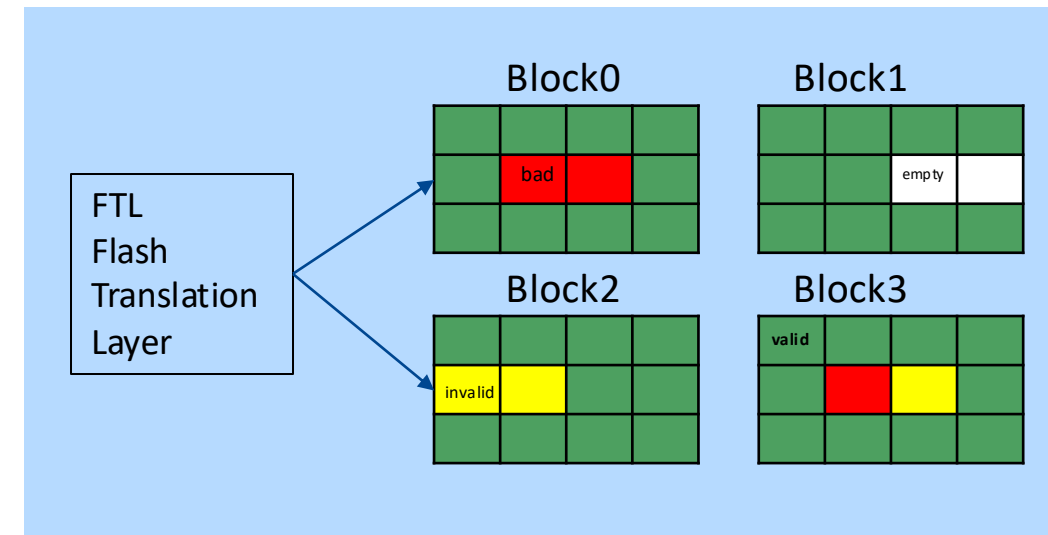
- Read and Write access to NVM occurs at '**page**' granularity (e.g 4-16 KB)
- Data cannot be overwritten, needs to be **erased** first
- **Erase** occurs at coarser granularity than 'page' – '**blocks**' that consist of multiple (e.g., 512) pages
- **Flash Controller** comes up optimizations to address the above limitations



Jin Y, Lee B. A comprehensive survey of issues in solid state drives. Advances in computers. 2019 Jan 1;114:1-69.

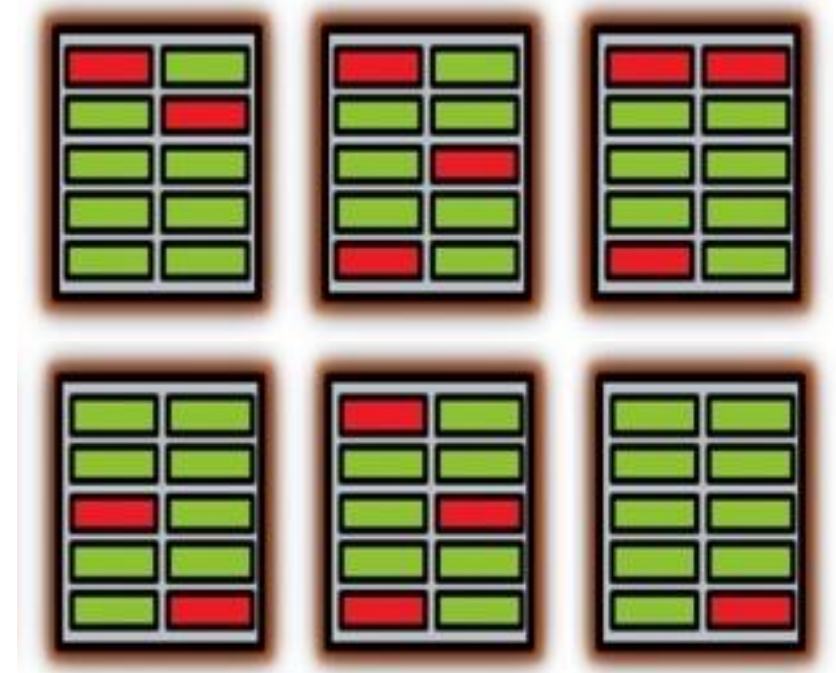
Case study: NAND Flash Memory

- **Flash Controller** maintains a flash translation layer (FTL) to track blocks
 - FTL can be implemented host- or device-side
- Empty (erased) pages can be used immediately
- If the data in a page is the current version, it is valid
- If the data in a page is out-of-date, it is invalid
- Invalid data needs to be overwritten
- If all pages in a block are invalid, the block can be erased



NAND Flash Memory - Writes

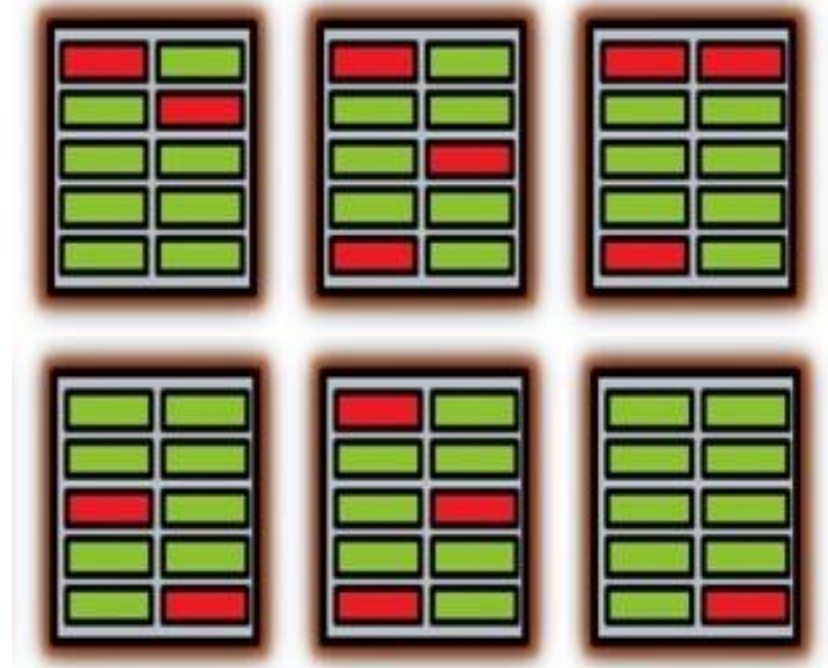
- For a Write request sent to a full SSD:
 - If there is an erased (valid) page:
 - combine the current data + new data into it
 - Mark the old page to be invalid



6 NAND **blocks** with valid (green) and invalid (red) pages. Each block has 10 pages.

NAND Flash Memory - Writes

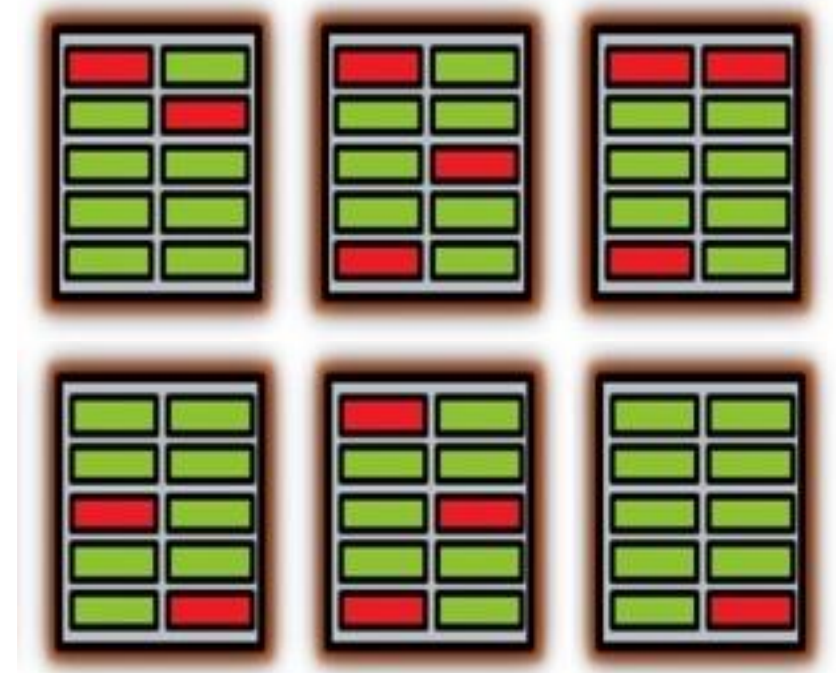
- For a Write request sent to a full SSD:
 - If there is an erased (valid) page:
 - combine the current data + new data into it
 - Mark the old page to be invalid
 - Or, if there is a block that can be erased, wait for erasure before writing to a page



6 NAND **blocks** with valid (green) and invalid (red) pages. Each block has 10 pages.

NAND Flash Memory - Garbage Collection

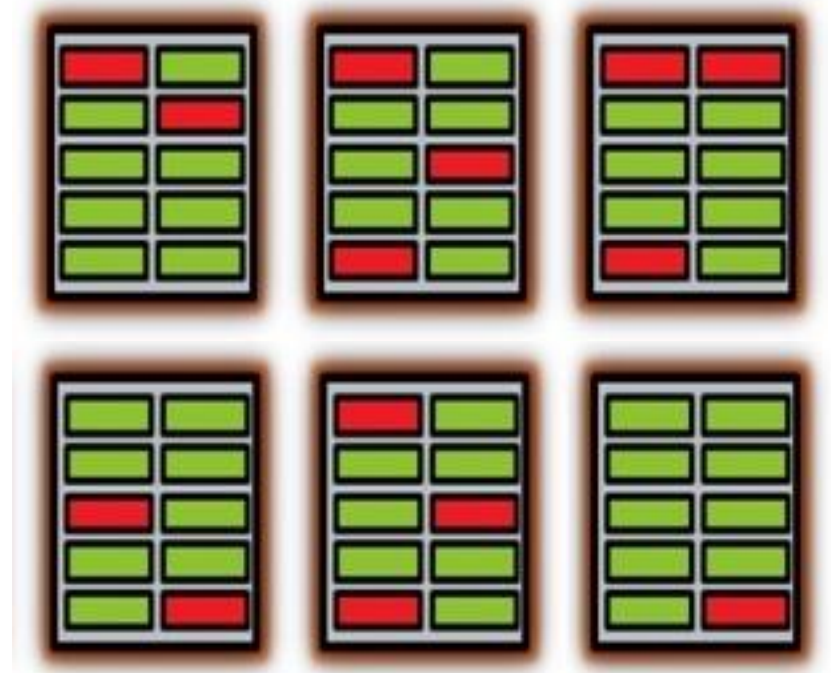
- For a Write request sent to a full SSD:
 - If there is an erased (valid) page:
 - combine the current data + new data into it
 - Mark the old page to be invalid
 - Or, if there is a block that can be erased, wait for erasure before writing to a page
 - Otherwise, we can 'merge' partial blocks that contain valid and invalid pages – **garbage collection**



6 NAND **blocks** with valid (green) and invalid (red) pages. Each block has 10 pages.

NAND Flash Memory - Garbage Collection

- Garbage Collection is performed by the controller
 - Merge valid pages into one block
 - Merge invalid pages into another block so that the whole block can be erased
 - Need a **staging space** to do so – a set of pages set aside by NVM device
 - Need to copy blocks that need to be merged into a staging space first - Latency
 - Then copy back to internal blocks – more Latency



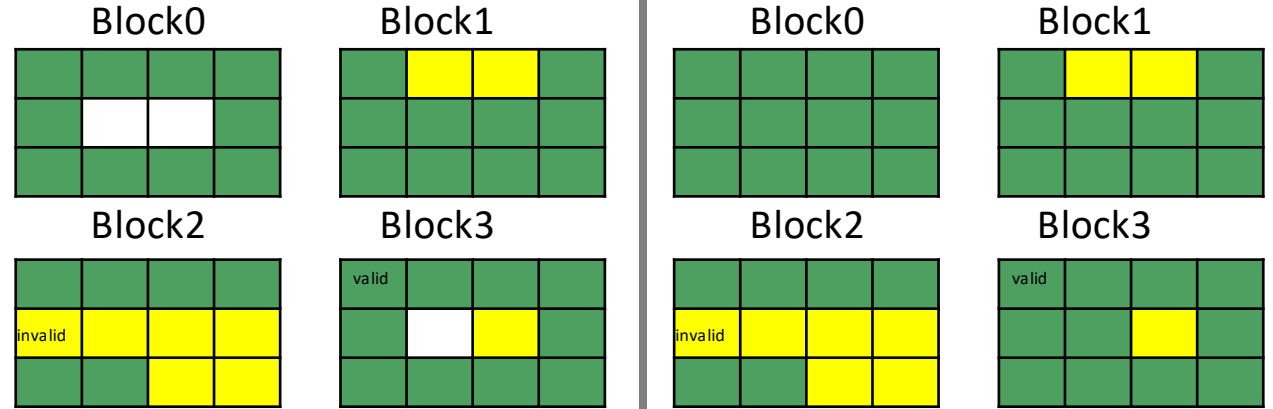
6 NAND **blocks** with valid (green) and invalid (red) pages. Each block has 10 pages.

Write Amplification

- NVM devices perform **Garbage Collection and Space Management** internally
- One write request can cause more than a page write (the data)
 - one or more page reads (by garbage collection)
 - one or more page writes (copy good data from the garbage-collected blocks)
- With these internal steps, a Write Access can be much slower, energy consuming than Read

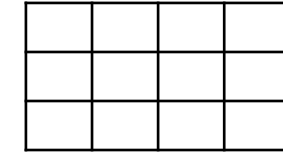
Example

Initial State

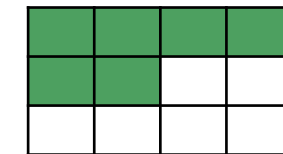


- {Read, Read, Write, Write, Read, Write} + Write
 - Read Latency
 - Read Latency
 - Write latency
 - Write Latency
 - Read Latency
 - Write Latency
 - Now Garbage Collection is needed:
 - Copy latency * 6 pages in Block 2 to staging buffer
 - Erasure Latency
 - Copy latency * 6 pages from staging buffer to Block 2
 - Write Latency
- Assume infinite staging buffer

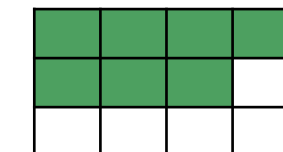
After Erasure the Block 2



After copy 6 pages to Block 2



After the last write





Outline

- Storage Devices
- Hard disk drives (HDD) Scheduling
- Non-volatile memory (NVM) Scheduling
- **Disk Attachment**
- RAID Structure



Disk Attachment

- Host-attached storage
- Network-attached storage (NAS)
- Storage-area network (SAN)

Host-Attached Storage

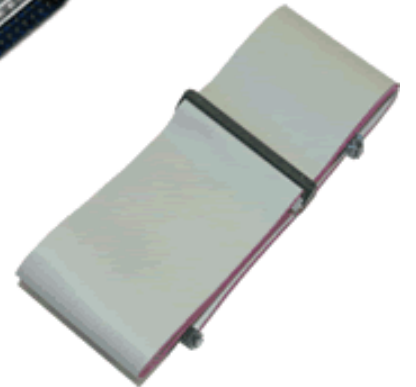
- Host-attached storage accessed through I/O ports talking to I/O buses.
- IDE or SATA support max. two drives per I/O bus.
- SCSI, up to 16 devices on one cable.
- Fiber Channel (FC) is high-speed serial architecture.



SATA Cable



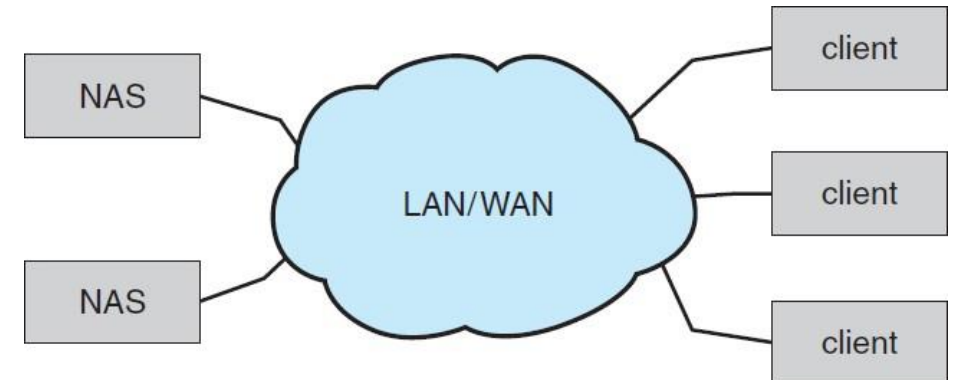
SCSI cable



IDE Cable

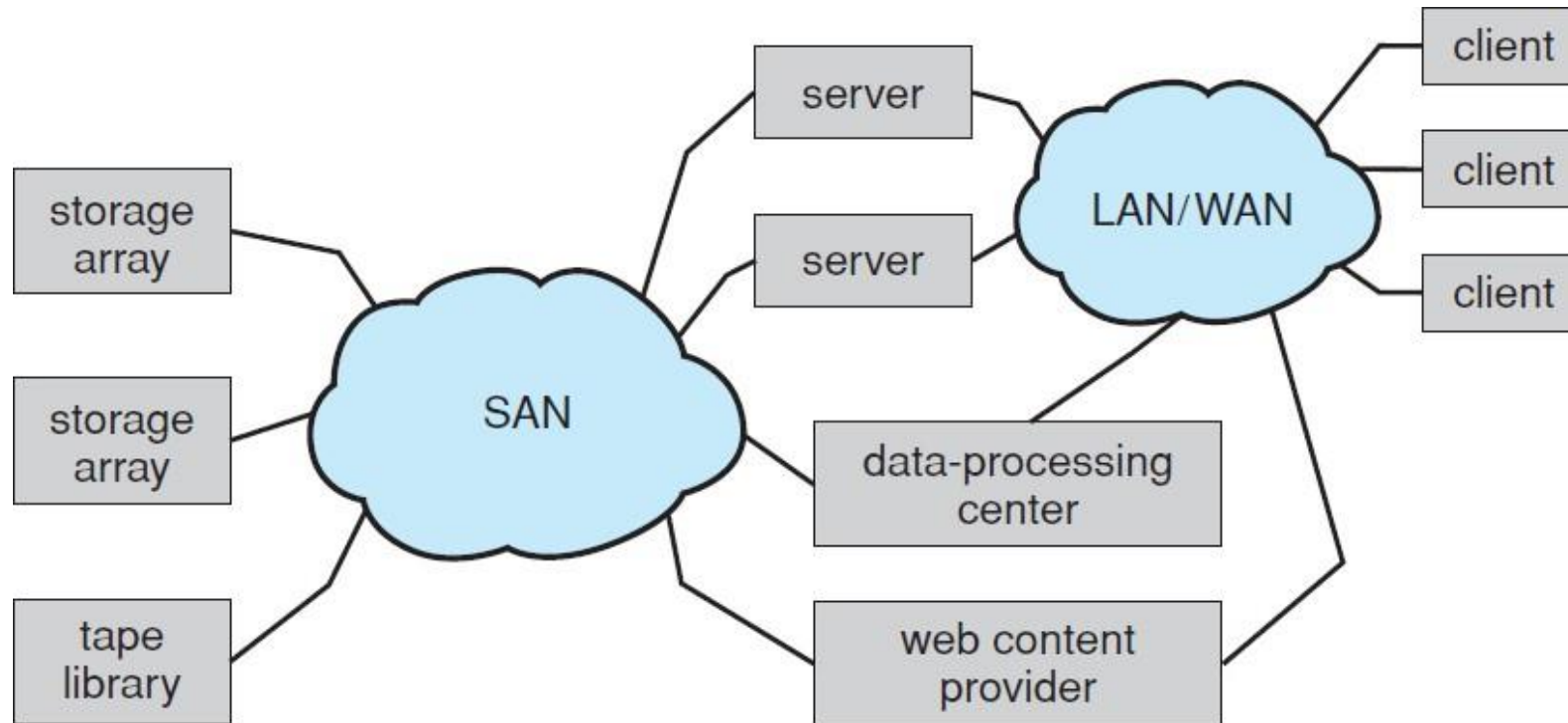
Network-Attached Storage (NAS)

- Network-attached storage is storage made available over a network.
- Remotely attaching to file systems.
- **FTP**, NFS and SMB are common protocols.

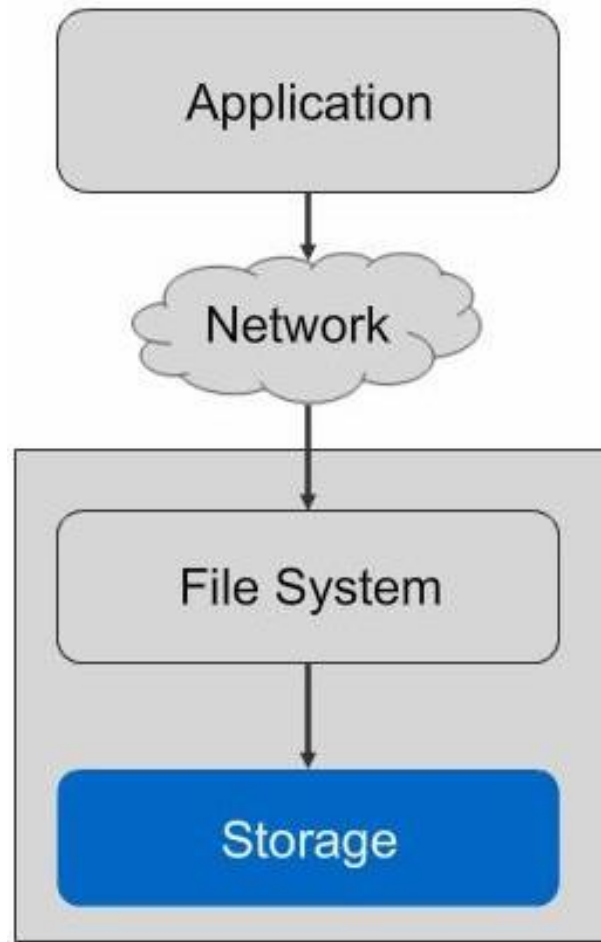


Storage-Area Network (SAN)

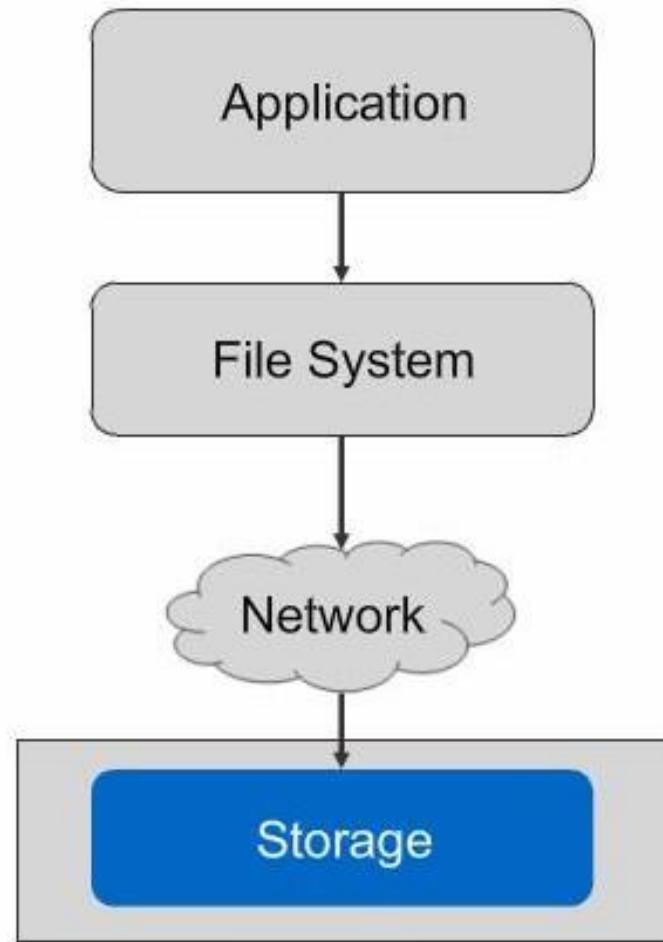
- Storage-area network is common in large storage environments.
- Multiple hosts attached to multiple storage arrays.



NAS vs. SAN



NAS



SAN

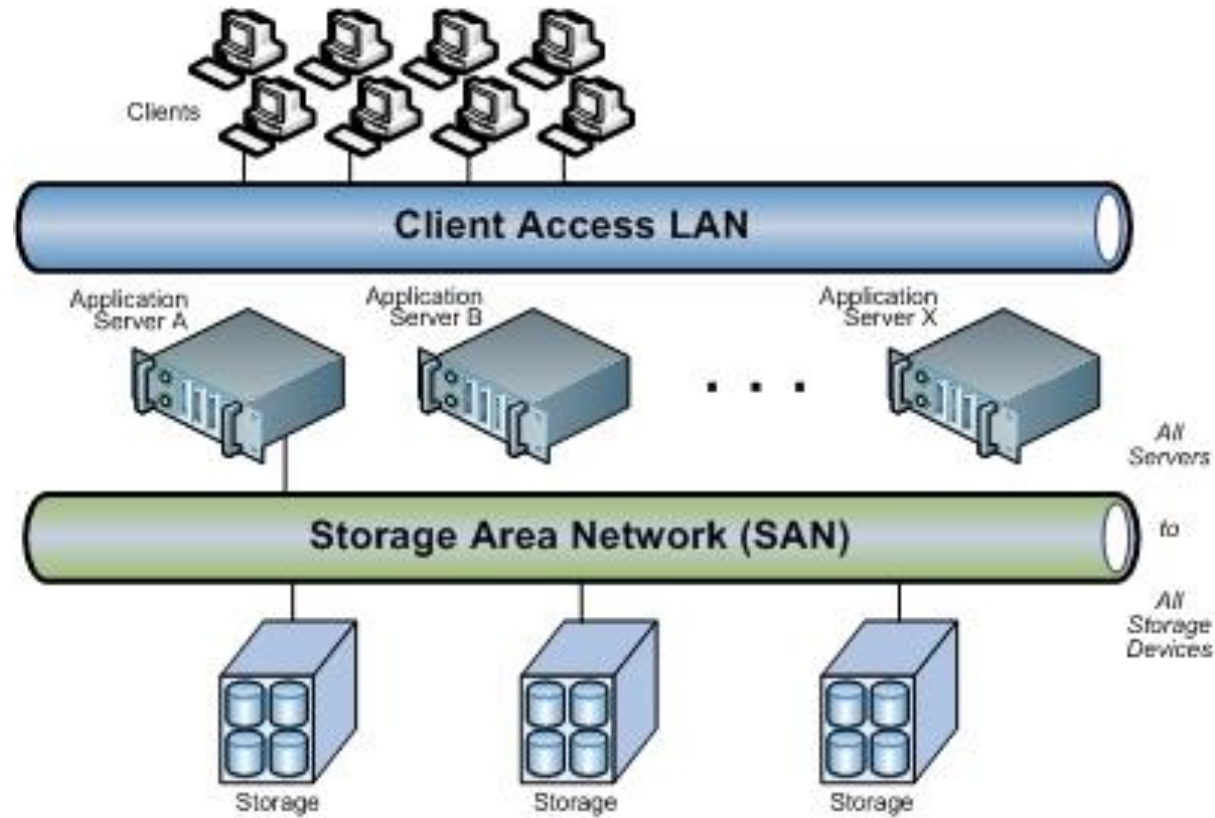


Outline

- Storage Devices
- Hard disk drives (HDD) Scheduling
- Non-volatile memory (NVM) Scheduling
- Disk Attachment
- **RAID Structure**

RAID

- RAID stands for **redundant array of independent disks**



Failure and Reliability

- Multiple disk drives provides reliability via redundancy.
- Increases the mean time to failure.
- E.g., if the mean time to failure of a single disk is 100,000 hours.
- The **mean time to failure** on an array of 100 disks will be $100,000/100 = 1,000$ hours, or 41.66 days
- It is not long at all.

Error Detection and Correction

- **Parity Bits** are used for detecting single-bit errors in memory
- Each byte has one associated parity bit
 - If one bit is damaged, the parity bit won't match the computed parity
- **ECC** (error-correction code) can detect and repair an error
- Disks drives use per-sector ECC, flash drives per-page ECC
- Need extra storage space

Data Mirroring

- The simplest approach to introducing redundancy is to **duplicate every disk**, called mirroring.
- A logical disk consists of two physical disks, and every write is carried out on both disks.
- If one of the disks in the volume fails, the data can be read from the other.
- Very expensive!



Data Striping

- Improves disk performance via **Parallelism**
- Use a group of disks as one storage unit.
- **Bit-level** striping: splitting the bits of each byte across multiple disks. E.g., with n disks, bit i of a file goes to disk $(i \bmod n) + 1$
- **Block-level** striping: blocks of a file are striped across multiple disks.
- Pros and Cons?

Disk1:	1 0 1 1 0 ...
Disk2:	0 1 0 1 1 ...
Disk3:	1 1 1 0 0 ...

Disk1:	Block A1	Block B1
Disk2:	Block A2	Block B2
Disk3:	Block A3	Block B3



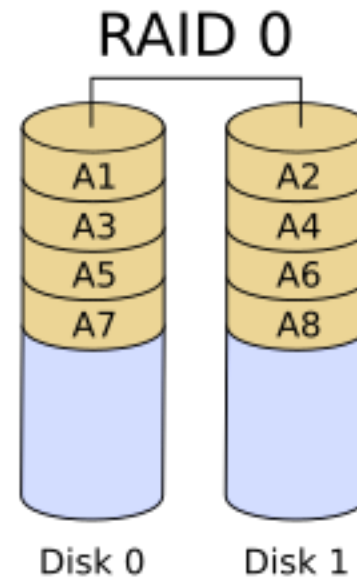
RAID Levels

- RAID: redundant array of inexpensive disks
- RAID schemes improve performance and reliability of the storage system by storing redundant data.
- RAID is arranged into 6 levels.

RAID Level 0

- Data (block-level) stripping
- Disk arrays with striping at the level of blocks but without any redundancy.

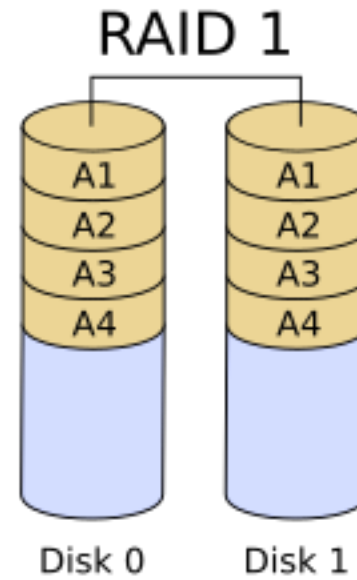
File 1:



RAID Level 1

- Disk Mirroring: A1 – A4 are duplicated on two disks

File 1:



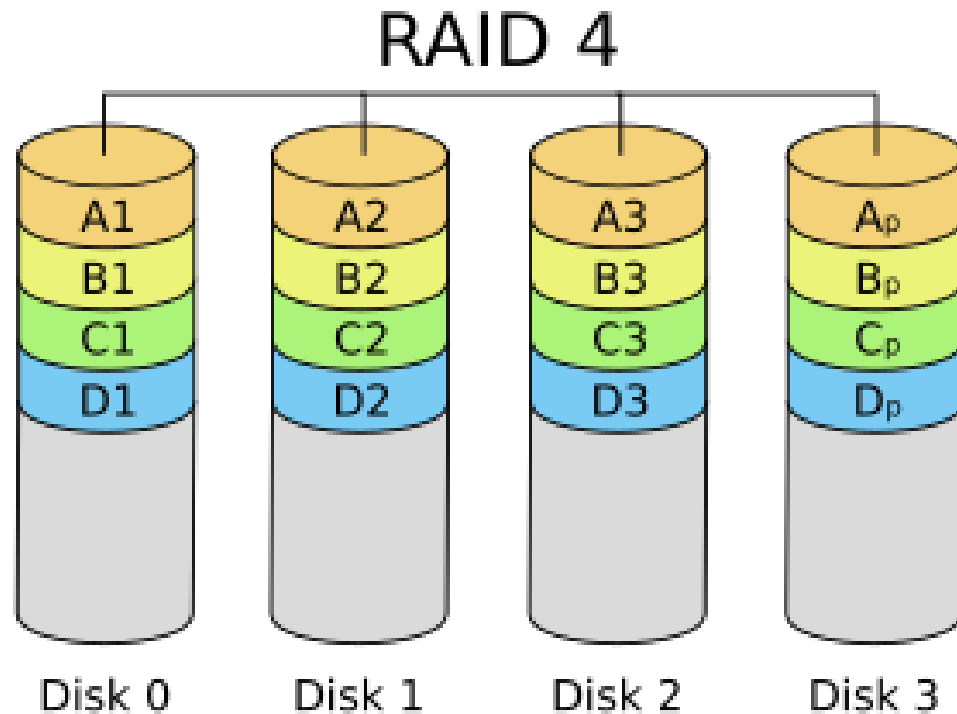
RAID 2 and 3 are Uncommon

- RAID 2 uses bit-level stripping + Parity
- RAID 3 uses byte-level stripping + ECC

Disk1:	1 0 1 1 0 ...
Disk2:	0 1 0 1 1 ...
Disk3:	1 1 1 0 0 ...
Parity:	P P P P P ...

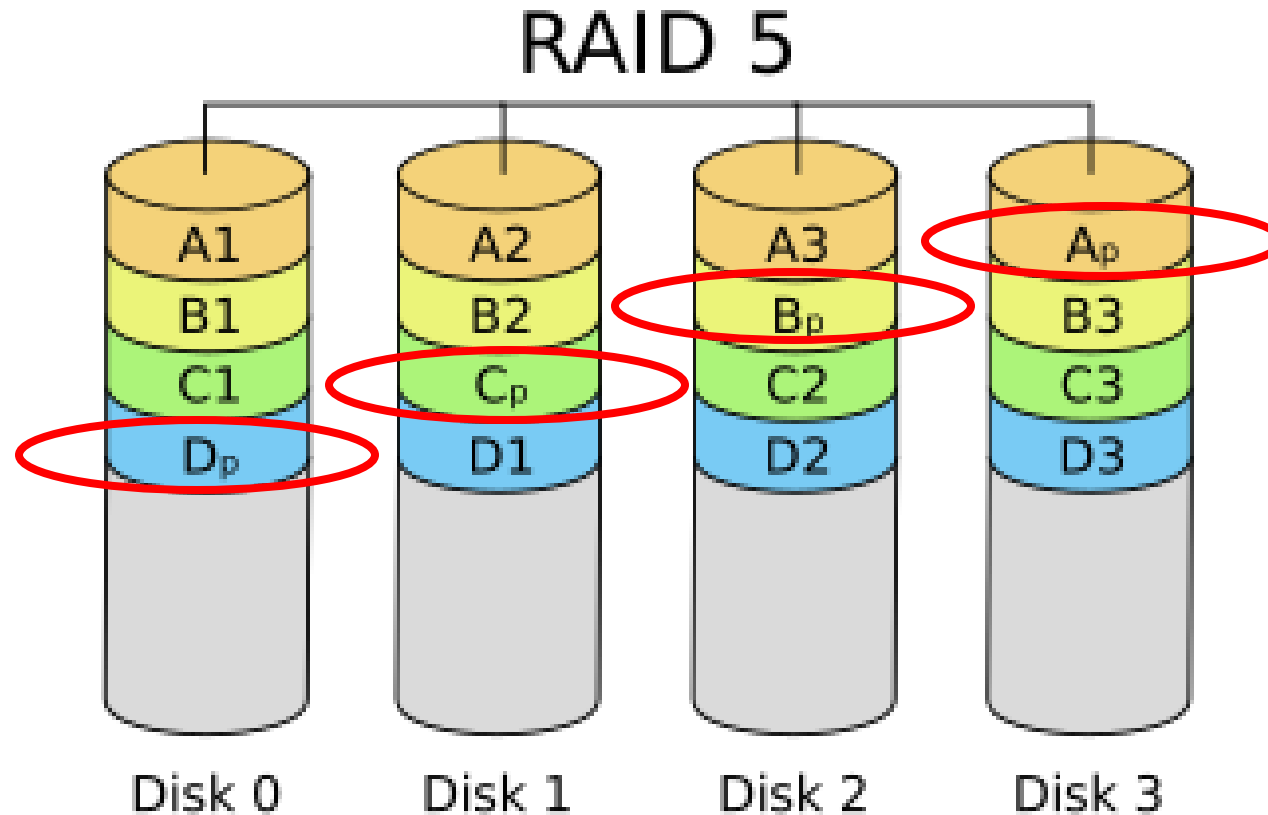
RAID Level 4

- Block-level striping, as in RAID 0.
- Include Error-correcting code (ECC), can detect and correct errors
- Keeps ECC on a separate disk (Disk 3 in this example) for corresponding blocks from N other disks.



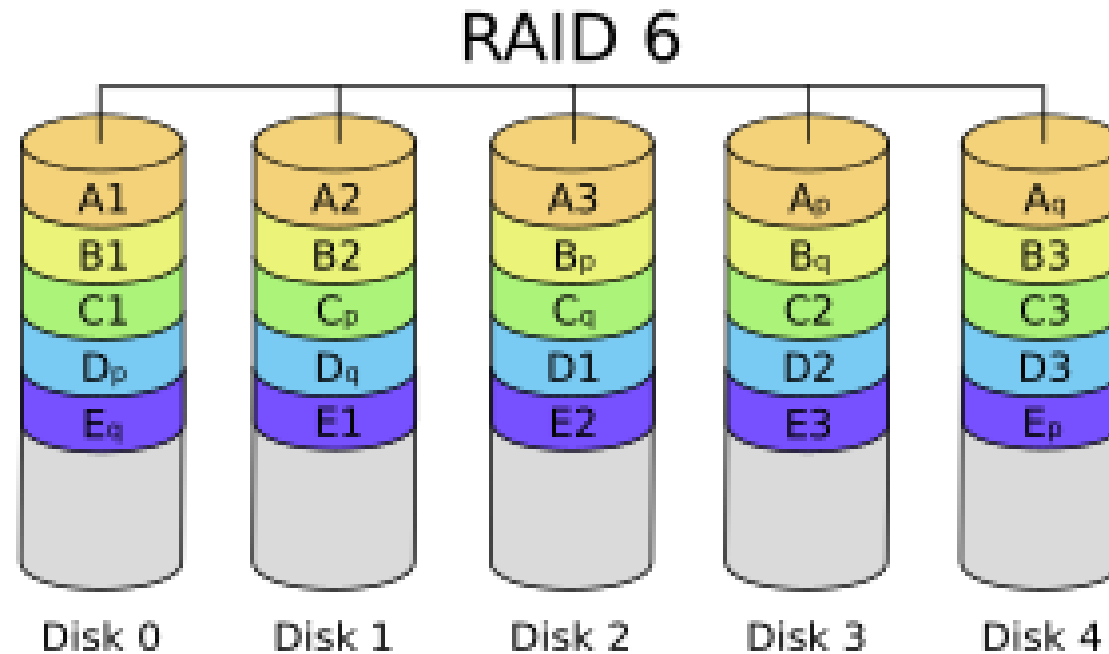
RAID Level 5

- Spreads data and ECC among all N+1 disks, rather than storing data in N disks and parity in one disk.



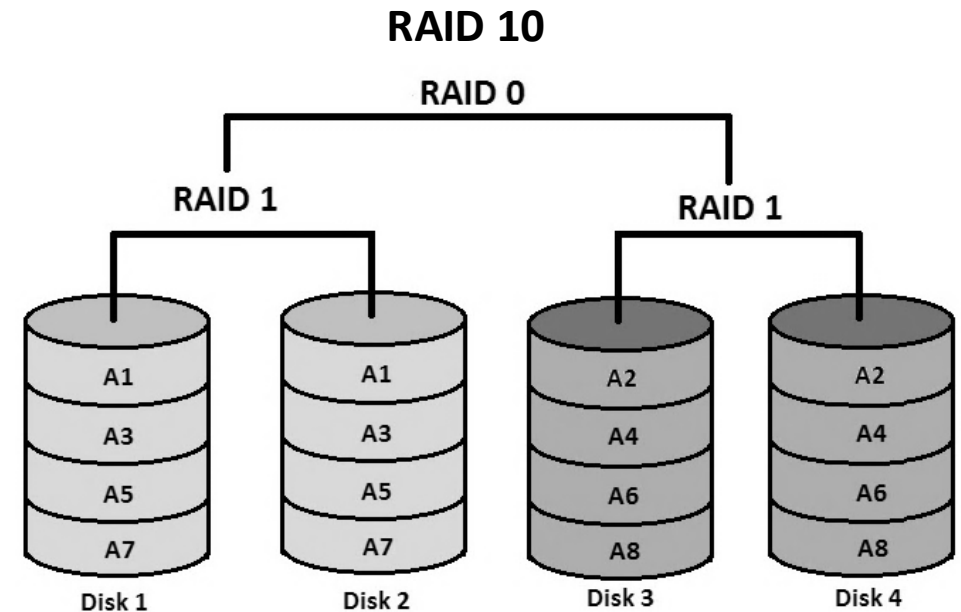
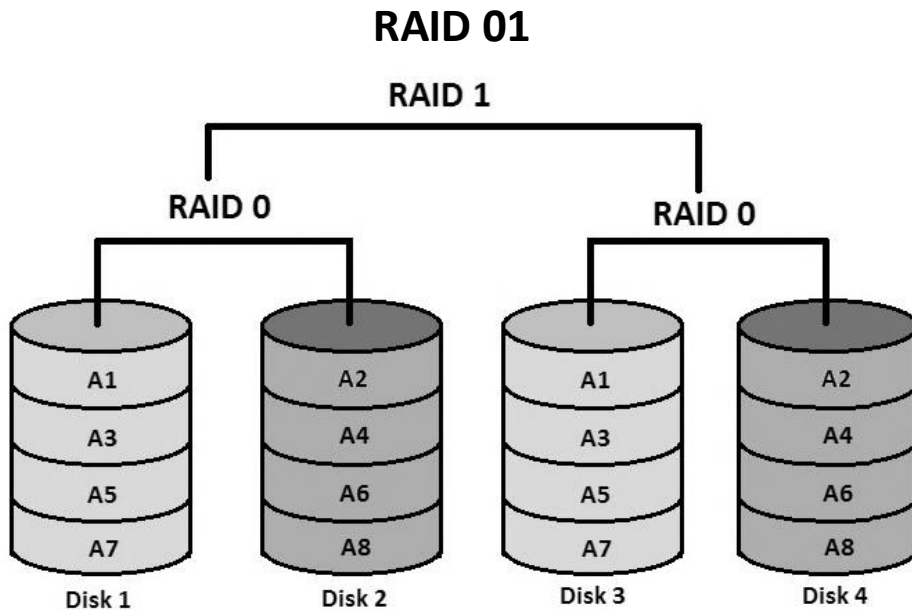
RAID Level 6

- Like RAID level 5 but stores 2 parity blocks instead of 1, spread over disks
 - Need at least 4 disks (at least 2 for block-stripping, 2 for parity blocks)
- The redundant information to guard against multiple disk failures.



RAID Level 0+1 vs. RAID Level 1+0

- Represent a combination of RAID 0 (stripping) and RAID 1 (mirroring)
- RAID 01: stripping then mirroring
- RAID 10: mirroring then stripping



Select a RAID Level

- **Cost** as in space overhead,
 - e.g., RAID 1 needs 100%, while RAID 4 and 5 need less
- Performance as in **Rebuild** Time,
 - e.g., RAID 1 is the fastest, simple data copy
 - E.g., RAID 5 takes a long time to recompute data



Summary

- Storage Devices: NVM, HDD
- HDD Scheduling: head movement
- NVM Scheduling: garbage collection
- Disk Attachment
- RAID Structure: cost, parallelism, reliability